Full-Stack Group Lab 1 Documentation

Violet French, Jarrod Hutcheon, Brayden Hermanson, & Isaiah Guilliatt

October 25, 2026

Contents

1 State Management - Violet **3**
- 1.1 Products . . . 3
- 1.2 Filtered-Products . . . 3
- 1.3 Search Filter . . . 3
- 1.4 Select Filter . . . 3

2 CRUD Psuedocode - Brayden **3**

3 Developer Log **3**
- 3.1 Violet . . . 3
  - 3.1.1 Filtering . . . 3
  - 3.1.2 Product Grid . . . 4
  - 3.1.3 Testing . . . 4
- 3.2 Brayden . . . 5
  - 3.2.1 CRUD Implementation . . . 5
  - 3.2.2 Product Tile . . . 5
- 3.3 Jarrod . . . 5
  - 3.3.1 JSON File . . . 5
- 3.4 Isaiah . . . 5
  - 3.4.1 Interface Design . . . 5

4 Team Jobs **5**
- 4.1 Violet . . . 5
- 4.2 Isaiah . . . 6
- 4.3 Brayden . . . 6
- 4.4 Jarrod . . . 6

5 Testing **6**
- 5.1 Test Cases . . . 7
  - 5.1.1 Fetching Products . . . 7
  - 5.1.2 No Match Checks . . . 8
  - 5.1.3 No Filter Check . . . 8

5.1.4 Shallow Sort Validation 9
5.2 Debugging 9
5.2.1 Filtering - Violet 9
5.2.2 Sorting - Violet 11

List of Figures

1 State Management - Violet

1.1 Products

The products state is the central state of the program. Products is stored in App.jsx and contains all of the products for displaying.

1.2 Filtered-Products

The filteredProducts state stores the filtered products which is given out to the ProductTile components for displaying. Filtered products is changed when the filter inputs change or when the overall products change causing it to re-filter the products. The filtered products are what gets displayed to the user even if no filtering is currently active. Due to filteredProducts being used by the product tiles it is also stored in App.jsx where the product grid is at. Each of the filtered products gets passed as a prop to product tiles via a map.

1.3 Search Filter

Search filter stores the current text search query. When the input for the search input changes it causes the search filter to change which also causes a rerender of the product tiles to match the new query. The search filter contains the raw text of the search box and assessing the queries is done by the filterProducts function. Queries can use relational operators (>=, <=, >, <, =, ==) to check price or they can use '&' as a delimiter for multiple combined queries. This filter is stored in App.jsx.

1.4 Select Filter

The Select filter acts very similar to search filtering but provides a more accessible interface for filtering. Rather than using a text query to filter the user just clicks on an option in a dropdown box of categories for product. When the state of the dropdown changes the select filter does too triggering a rerender of the product tiles. This filter, like searchFilter, is stored in App.jsx.

2 CRUD Psuedocode - Brayden

3 Developer Log

3.1 Violet

3.1.1 Filtering

To implement the filtering I decided to copy the style I've used in the past few programs. I used both a category select option and a text search filter. When filtering

the products it applies both of these to the list of products and returns the filtered list. I also added alphabetical sorting like in my previous programs. I originally designed the filtering as its own component called FilterForm but realized near the end that this would not work very well because of how React state works. Instead I moved it up to App.jsx and redesigned the heirarchy. This wasn't a problem as I did this on my own time when the component heirarchy was still in its infancy. Going into this project we didn't have enough time to plan things out and the FilterForm is when I started to realize this. After setting everything up I cemented a component heirarchy diagram to guide further features now that we had a better idea of what our product will look like.

I wasn't happy that I had to move the logic back to App since I know it will make things very messy in the long run. With me moving it back to App it resulted in around 150 lines of code for something that would be far better as a component. There may be a way to do it as a component but it would be such a headache to set up that it would wind up making things more complex. This is definitely a pitfall of React. The increased complexity in writing dynamic code hurts how modular you can make the program. I do think with a database things would be a lot better since database langauges like SQL include filtering clauses and sorting clauses which would trivialize this. So in all honestly you can't blame React too much but I must say that state is a headache to deal with as compared to something like classes and properties you might be working with if you were using a basic templating engine such as Thymeleaf (less reactive but is also SSR)

3.1.2 Product Grid

The product grid underwent many forms. Initially it was slated to be a React component but I decided not to do that since I didn't see the point. This later changed when I realized it wasn't updating when filtered products updated. This motivated me to turn it back into a component so that when filteredProducts changed so to would the product grid. I also wanted to add a conditionally rendered span that would let you know if there were no matches. I initially placed this on the App level but this causes problems. Once the span triggers then it will never stop triggering. This is because the span isn't tied to any kind of state so once it triggers React will never need to rerender it meaning it will never attempt to mount the product grid even when you change the filters back. The solution is to move the span into the product grid component. With this, anytime the filteredProducts state variable changes it will rerender the ProductGrid. Anytime it renders the product grid it will check if there are any products matching and display them if there are or display the span if there aren't.

3.1.3 Testing

I was in charge of writing most of the testing code. I was not initially going to make robust test cases but I realized as I was testing that simple checks here and there were not sufficient and we needed a stream-lined was to do this. I did this by writing a test() function inside App.jsx that would test different parts of our code. Within this function there is a arrow function called fAssert() which is used to speed up assertion checks. It takes in three parameters: value, expected, msg. Value is the value you want

to do the assertion check on and is the only required parameter. This is the value you want to compare to expected to see if it matches. Expected is the value you expect to get, if the value does not match expected there is an error that gets logged in the console. Expected has a default value of true meaning you can pass a conditional to value to quickly do an assertion check. The final parameter, msg, outputs a unique message when assertion fails. This msg will follow the text 'Assertion Fail: '.

The fAssert() function was originally called fassert since it would flushSync than assert. Flushing the sync is used to force a rerender meaning you will get the most recent state variables. Flushing sync was recommended by the React documentation for immediately getting an updated state. This did not work however since the code only runs on first render (mounting) so flushing the sync doesn't do anything. The solution was to instead directly set the state variables skipping past the state setting function. This is used in this case since we don't need a rerender we just need to update the value of the states that way it can be used internally for the assertion checks. Despite flushing being removed from the function I kept the name fAssert incidentally after doubling back on my code several times and just decided to keep the name. Below is the code for the fAssert() function.

```
const fAssert = (value, expected=true, msg="Assertion test failed") => {
    if (value !== expected) console.error("Assertion Fail: " + msg);
}
```

The test() function is called once on mount using an effect. This effect is only triggered if the variable TESTING_ENV is set to true otherwise testing will not occur. A few functions were redesigned during creation of test cases to allow them to be used on first mount.

Information about the different test cases is outlined in a later section.

3.2 Brayden

3.2.1 CRUD Implementation

3.2.2 Product Tile

3.3 Jarrod

3.3.1 JSON File

3.4 Isaiah

3.4.1 Interface Design

4 Team Jobs

4.1 Violet

Violet was in charge of creating the filtering and searching functions. She also created much of the doucmentation including: this document, state Management Doc-

ument, Component Heirarchy Diagram, and the README. Violet also served as the group leader delegating tasks among team members and herself and keeping people on track. Violet created the product grid which links together the filtering features with Brayden's product tiles. Violet wrote many test cases for the code and the function that runs those tests.

4.2 Isaiah

Isaiah was in charge of web design and had the final say in how the page looked. Isaiah was primarily in charge of the CSS but was also responsible for the creation of static elements like the header and footer for the site. He created the interface sketch and built the site based off that plan.

4.3 Brayden

Brayden was in charge of creating the product tiles and CRUD functionality. Brayden also wrote the psuedocode for the CRUD operations which is present in this document.

4.4 Jarrod

Jarrod was in charge of the data for the application. Jarrod created the products in the JSON file and made sure they were pulled in properly. Jarrod structured the JSON file in a preplanned format so that other people knew what properties to expect.

5 Testing

Most of the testing for this program was done via the test() function in App.jsx. The test function gets run on mount via a useEffect(), this is to prevent it running multiple times. Many functions had to be redesigned in ways that would allow them to run without the state variables. State variables can be used for testing but they cannot be set using their setState function. The setState function works lazily in that it will only change state on next render. Seeing as the useEffect with an empty dependency array only runs on first mount, when the component is first being rendered these setStates will never work withing the context of the test function. State variables can be used but they must be set directly rather than via the setState function provided by useState(). Typically you should avoid this as the purpose of useState is to create variables who can trigger rerenders of dependencies but since the test function is based on assertion tests and isn't concerned with rendering this is alright. Additionally the products fetching function had to be created as the test function must be able to pull in the products to run its tests. To trigger testing the TESTING_ENV variable must be set to true. This is not an environment variable unlike its name may suggest, instead it sits in the code directly and must be edited by the programmer to enable or disable it.

The different test cases utilize some shared variables and functions listed below

Listing 1: Persistent variables of test()

```
const fAssert = (value, expected=true, msg="Assertion test failed") => {
    if (value !== expected) console.error("Assertion Fail: " + msg);
}

const prods = await fetchProducts();
let filtered;
```

5.1 Test Cases

This section will go over the various test cases conducted.

5.1.1 Fetching Products

A test case was written to test the fetching of products from the JSON file. This test case required that the previous logic for fetching products be changed. Before the test case the fetching of the products was written once inside a useEffect, this works but it isn't reusable. Since state doesn't change until the next render we can't rely on a useEffect to update the state with a setState like it previously had been. Instead the logic for fetching products was extracted into an async function. This allowed the logic of the useEffect to remain the same but instead of chaining the Promise returned by the fetch it instead just used one then clause to set the states and then catches any errors that might of been tossed by the function. The function now grabs the data from the JSON instead meaning it can be used in multiple places of the code. This allows the test() function to get a hold of the products before first render by calling the function. Using this we pull in the products from the JSON in our test() function and assign it to a variable. Then to run a basic test we check to see if the prods variable has a length greater than 0. This doesn't check that we got all the products or even the right ones but it checks that we got something. Checking if we got the right products would mean hard-coding the right values into the test function which creates tight coupling between products.json and the test() function which is undesirable and harms our ability to scale.

Listing 2: fetchProducts() function

```
/**
* Fetches the products from the JSON file
* @returns The products fetched and formatted
*/
async function fetchProducts() {
    const response = await fetch("/products.json");

    if (!response.ok) throw new Error("Failed to fetch products");

    const json = await response.json();

    return json.map(prod => ({
```

```
        ...prod,
        productCategories: prod.productCategories.map(cat => cat.toLowerCase())
    }));
}
```

Listing 3: Test Case

```
const prods = await fetchProducts();
fAssert(prods.length > 0, true,"Products did not pull in correctly");
```

5.1.2 No Match Checks

Two test cases were written to check if the filtering would properly return no products when filtering for something with no matches. Both checks relied on an obscure character sequence that would be unlikely to be found in any part of the products. Ideally we would prevent these characters from being used but for now this obscurity should work fine. These characters are so obscure they cannot be plain typed into this document as they are not UTF-8 characters, in this document they will be referred to as 'OBSCURECHARACTERS'

The first of two checks was a category select check. By setting the categorySelected to the obscure sequence of characters then running filterProducts we will get the products filtered by that category. A successful test is one in which no products are returned

```
// CHECK FOR NO CATEGORY MATCH
searchText = "";
categorySelected = "OBSCURECHARACTERS";  // Shouldn't be any categories for this
filtered = filterProducts(prods);
fAssert(filtered.length, 0, "Returned products for invalid category filter");
```

The next check was the same idea but it was conducted on the search text instead of the category

```
// CHECK FOR NO SEARCH MATCH
searchText = "OBSCURECHARACTERS";  // Such an obscure character combo no one wou
categorySelected = "all";
filtered = filterProducts(prods);
fAssert(filtered.length, 0, "Returned products for invalid search filter");
```

5.1.3 No Filter Check

A test case was written to see if when no filters were applied the filterProducts() function would return all products. A successful test is one in which the products from the JSON file have the same length as the products after filtering. We cannot check for equality on this as filterProducts() also alphabetizes the products meaning they do not preserve order.

```
// CHECK FOR NO FILTERING
categorySelected = "all";
searchText = "";
filtered = filterProducts(prods);
fAssert(filtered.length, prods.length, "Did not pass check for no-filter");
```

5.1.4 Shallow Sort Validation

A test case was written to test whether filtered products were being alphabetized. This check is shallow and does not check beyond the first character or for string length concerns. This test has been deemed sufficient for most applications and a more in-depth test may be written later if necessary. The code maps each product in filtered to the character code of its first character. A for loop then goes through the character code array and ensures that every character code is greater than or equal to the preceding ones by tracking the previous biggest code. A successful test is one in which no character following another has a smaller character code.

```
// CHECK ALPHABETICAL - LIGHT CHECK AND ONLY CHECKS FIRST LETTER
categorySelected = "all";
searchText = "";
filtered = filterProducts(prods);
filtered = filtered.map(prod => prod.productName.charCodeAt(0));
let biggest = 0;

for (const cc of filtered) {
    fAssert(cc >= biggest, true, "Light check on alphabetic order failed");
    biggest = cc;
}
```

5.2 Debugging

5.2.1 Filtering - Violet

Upon writing tests for the filtering many errors were found in the filtering code. The first error encountered was that the category select filter was not being applied. The problem was a simple one, the code wasn't storing the filtered array after category filtering. The Arrays.filter() method does not mutate the original array instead returning a new array with the filter applied. My mistake was not setting filteredProducts equal to this new value. The new code after the fix is listed below

```
if (category !== "all")
filteredProducts = filteredProducts.filter(
    prod => prod.productCategories.includes(category)
);
// Filter by category dropdown, productCategories should already be lowercase
```

Further testing of the filtering code revealed another error in which the filtering was not using the most recent filters instead using the previous ones. This was a case of state not updating. As mentioned in the Testing section, state updates lazily waiting till next rerender to reflect that new state. This can cause issues for states that aren't used by components such as the filters. This could be fixed in many way including adding the filters somewhere in the component to force it to rerender on their change. Instead I just opted to add a useEffect with the filter states as dependencies. The new code had the inputs on the page update the state of the filters which in turn triggered the effect causing it to refilter the products with the most recent data. The updated code is below

```
// Filtering stuff
let [filteredProducts, setFilteredProducts] = useState([]);
let [searchText, setSearchText] = useState("");
let [categorySelected, setCategorySelected] = useState("All");

/**
* Updates the state of the filters and then updates the filtered products trigge
* @param {Event} e The event that spurred this action
*/
function updateFilters(e) {
    switch (e.target.className) {
        case "selectFilter":  // Category selection changed
        setCategorySelected(e.target.value);
        break;
        case "searchFilter":  // Search query changed
        setSearchText(e.target.value);
        break;
    }
}

useEffect(() => {
    setFilteredProducts(filterProducts);
}, [categorySelected, searchText]);

/**
* Filters the products by the current filtering queries, alphabetizes the results
* @param {Array<Object>} prods Products to use instead of the state products, us
* @returns {Array[Object]} an array of the products filtered and sorted
*/
function filterProducts(prods=null) {
    // FILTERING CODE OMMITTED
    return filteredProducts.sort(
        (a, b) => alphabetCompare(a.productName, b.productName)
    );
}
```

5.2.2 Sorting - Violet

The filtering function performs and alphabetic sorting of the products by their name. This ensures the products are in order of their names ascending. The logic for this however did not work in initial testing. The error was that alphabetCompare, the function that tells Arrays.sort which item comes first, was being passed the product objects instead of the product names. This was a simple fix just needing to change it from passing the parameters a and b to passing a.productName and b.productName. The updated code is below

```
/**
* Compares two strings and returns a number indicating which comes first alphabe
* @param {String} a The first string
* @param {String} b The second string
*/
const alphabetCompare = (a, b) => {
    if (a === b) return 0;       // They are the same string so it doesn't matter

    for (let i = 0; i < a.length && i < b.length; i++) {
        if (a.charCodeAt(i) > b.charCodeAt(i)) return 1;
// b's letter comes first
        else if (a.charCodeAt(i) < b.charCodeAt(i)) return -1;
// a's letter comes first
    }

    // They have the same characters up to some point meaning we assess string l
    return (a.length > b.length) ? 1 : -1
}

/**
* Filters the products by the current filtering queries, alphabetizes the results
* @param {Array<Object>} prods Products to use instead of the state products, us
* @returns {Array[Object]} an array of the products filtered and sorted
*/
function filterProducts(prods=null) {
    // FILTERING CODE OMITTED
    return filteredProducts.sort(
        (a, b) => alphabetCompare(a.productName, b.productName)
    );
}
```